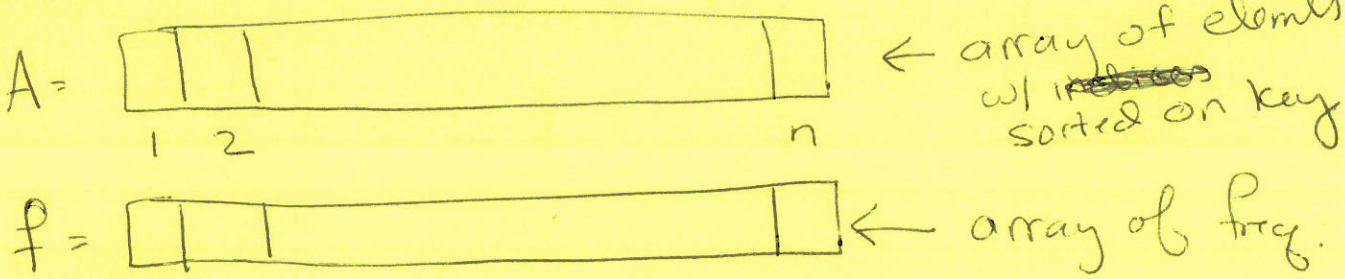
26 Feb 2026

Question:

In "regular" binary search, we pick the median as a pivot (= we have a balanced BST)
As a result, the worst-case runtime for a single query is $\Theta(\log n)$.
But, what if we want to do a lot of queries? Is a Balanced BST always the best?
In particular:

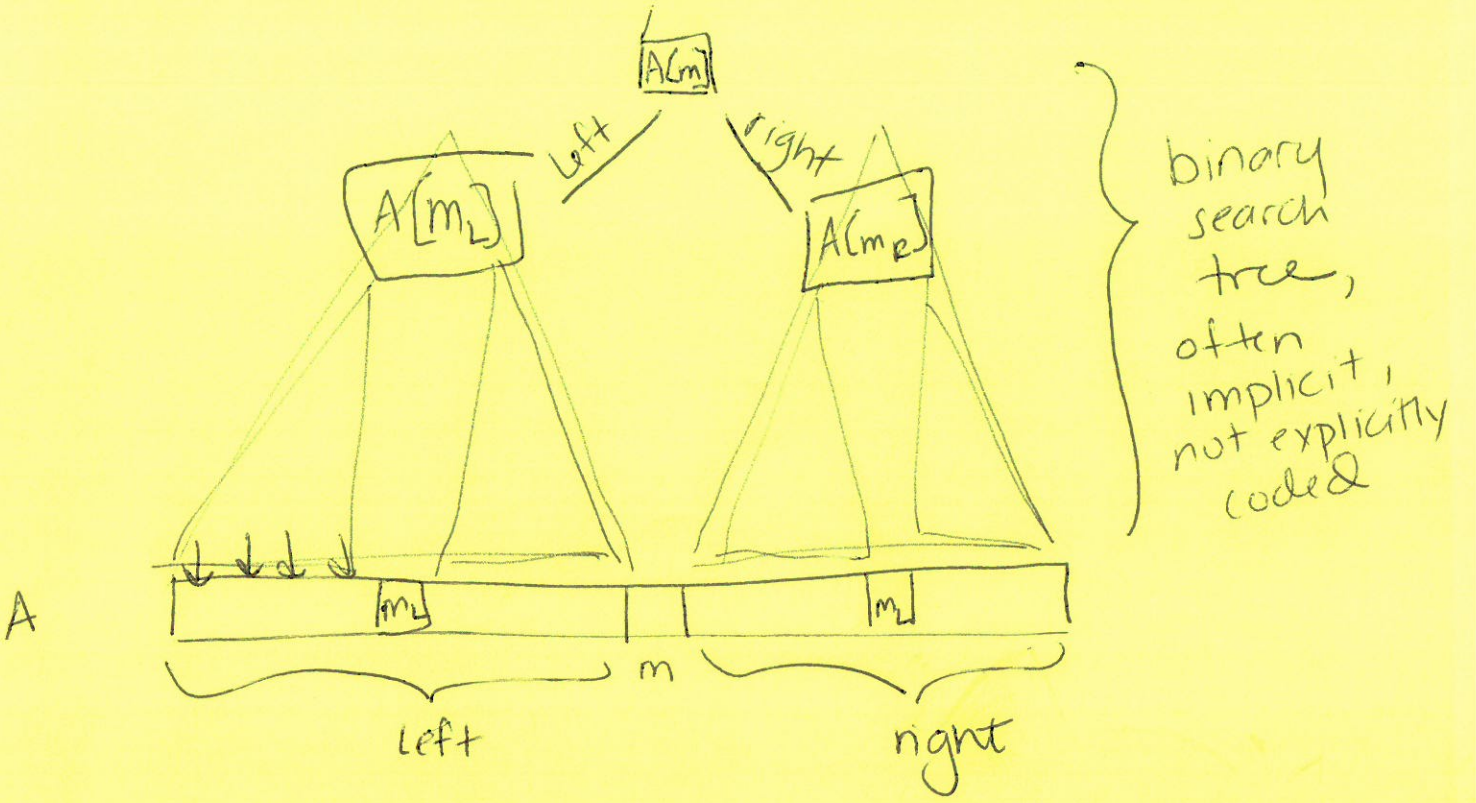


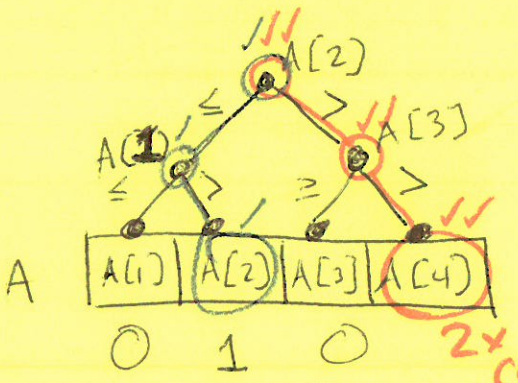
A[i] is searched f[i] times.

think of $A[i] \in \mathbb{R}$, so it is the key.

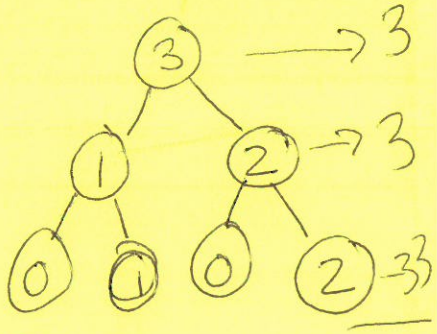
Q1 Give an example where balanced search does not do best.

Q2 Think about how can we solve this + how does it relate to recursion?



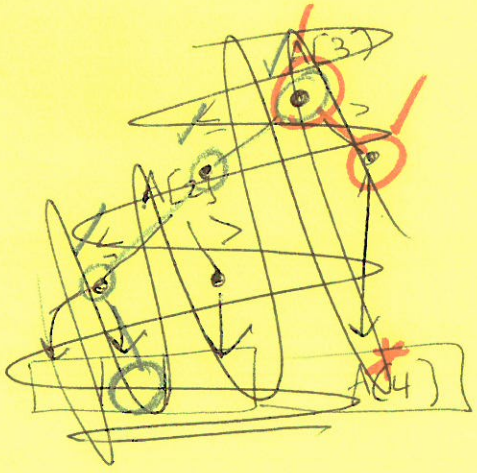


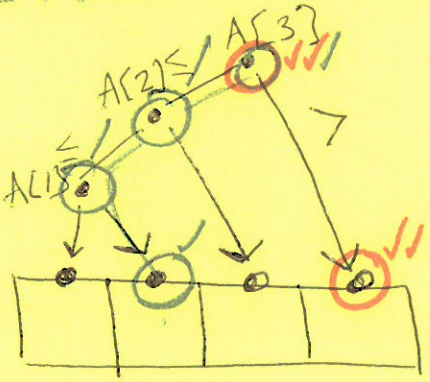
times each node visited across all searches



9

cost = 3 nodes × 2 visits





3 nodes visited
1
1
3

~ runtime

8 nodes visited

⇓

balanced was not best!

f	n	n-1	. . .	3	2	1

in Balanced tree,
visit $\Theta(\log n)$ nodes per element

$$\Rightarrow \sum_{i=1}^{n} \Theta(\log n) \cdot f[i]$$

(sum over all ele.)

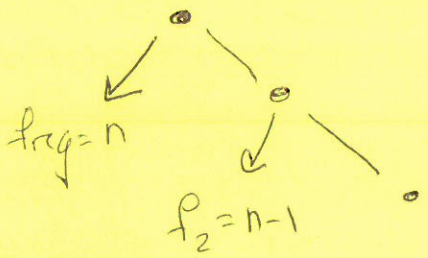
$$= \Theta(\log n) \sum_{i=1}^{n} n-i+1$$

$$= \Theta(\log n) \sum_{i=1}^{n} i$$

$$= \Theta(n^2 \log n)$$

---

unbalanced tree

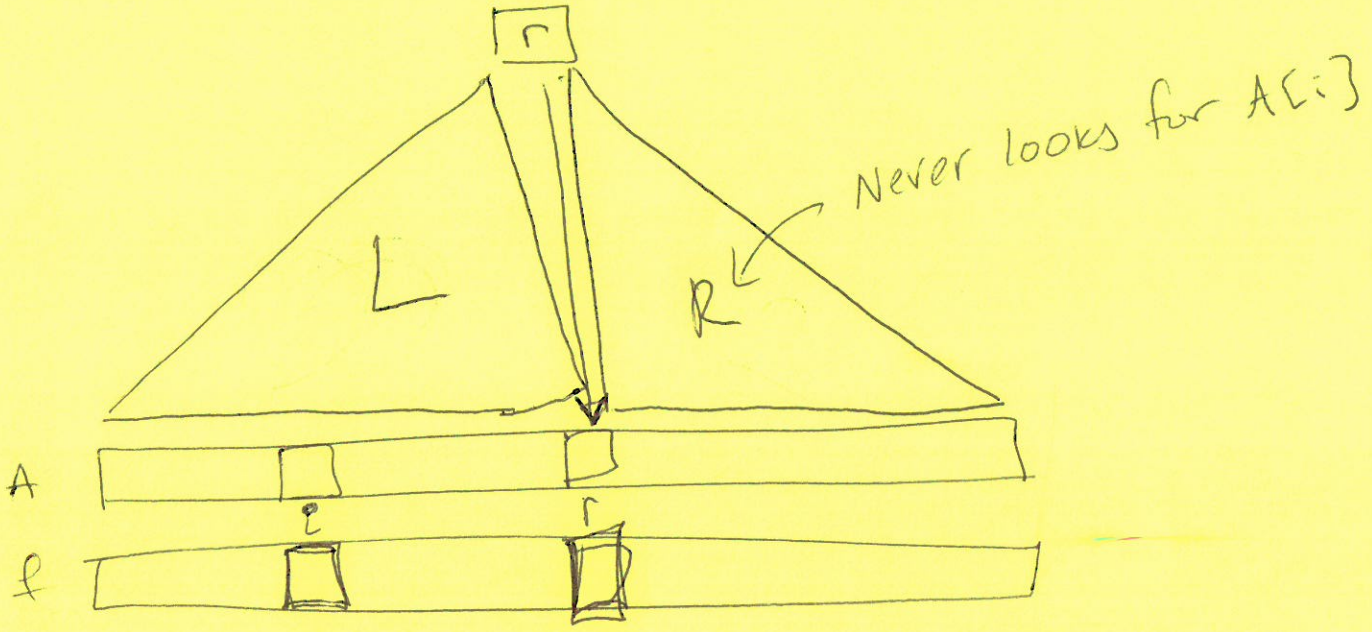


$$\sum_{i=1}^{n} f[i] \cdot (i+1) = \sum_{i=1}^{n} (n-i+1)(i+1)$$

(will be better for this type of example)

But ... greedy never works (according to JE)
So, something else for general case

Given a specific search tree,
consider the cost of search tree:



# of nodes looked at for A[i]
$= 1 \cdot f(i)$ for the root. + # of times searched in L

Sum over all i to get # of nodes visited (with repeats)

$$\sum_{i=1}^{n} \left(f[i] + \#\text{ times searched in } L + \#\text{ times searched in } R \right)$$

$$\text{Cost}(T) = \sum_{i=1}^{n} f[i] + \text{Cost}(L) + \text{Cost}(R)$$

Finding the best = try all trees = try all roots

$$\text{OPT COST}(A[1\ldots n], f[1\ldots n])$$

$$= \begin{cases} 0 & \text{, if } n = 0 \\ \sum_{i=1}^{n} f[i] + \min\limits_{1 \le r \le n} \left\{ \begin{array}{l} \text{OPT COST}(A[1\ldots r-1], f[1\ldots, r-1]) \\ +\text{OPT COST}(A[r+1\ldots n], f[r+1, \ldots, n]) \end{array} \right\} \end{cases}$$

↑ our big, lazy recursion. $\Theta(3^n)$

to avoid sending all data again, just use indices!

$\text{OPT COST}(i, k)$ ← initial call: $\text{OPT COST}(1, n)$

$$= \begin{cases} 0 \\ \sum_{j=i}^{k} f[i] + \min\limits_{i \le r \le k} \left\{ \begin{array}{l} \text{OPT COST}(i, r-1) \\ +\text{OPT COST}(r+1, k) \end{array} \right\} \end{cases}$$

Rather than be greedy, use DP

"smart recursion"

$$F(i,k) := \sum_{j=i}^{k} f[j] \qquad \text{cost} = \Theta(k-i)$$

How many possible values do we have here?

$i \in \{1, 2, \dots, n\}$, $k \in \{1, 2, \dots, n\}$

n^2 fcn values for F to evaluate

ex: $n=4$

digraph of dependencies

Smart Recursion:

(1) memo-ize = save values in ~~good~~ array as we go

(2) evaluate in a smart order.

"plucking off" to define as a recursion

$$F(i,k) = \begin{cases} f[i] + \sum_{j=i+1}^{k} f[j] & i \le k \\ 0 & \text{, if } i > k \end{cases}$$

$$F(i,k) = \begin{cases} f[i] + F(i+1,k) & , i \le k \\ 0 & , i > k \end{cases}$$

Given a DAG, $G=(V,E)$,

we can always find an ordering of the nodes such that $\forall (i,j) \in E$ node j appears before node i

(algo: topo sort)